

# Protocol Audit Report

Version 1.0

*Cyfrin.io*

October 26, 2026

# Protocol Audit Report

Carlos Matos

September 25, 2026

Prepared by: Carlos Matos Lead Security Researcher: - Carlos Matos

## Table of Contents


- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] Storing the password on-chain makes it visible to anyone, and no longer private.
    * [H-2] ‘PasswordStore::setPassword’ has no access controls, meaning a non-owner could change the password.
  - Informational
    * [I-1] The ‘PasswordStore::getPassword’ natspec indicates a parameter that dosen’t exist, causing the natspec to be incorrect.

| | | Impact | | |
|---|---|---|---|---|
| | | High | Medium | Low |
| | High | H | H/M | M |
| Likelihood | Medium | H/M | M | M/L |
| | Low | M | M/L | L |

## Protocol Summary

PasswordStore is a smart contract application for storing a password. Users should be able to store a password and then retrieve it later. Others should not be able to access the password.

## Disclaimer

The Carlos Matos team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

## Risk Classification

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

## Audit Details

**The findings described in this document correspond to the following commit hash:**

```
7d55682ddc4301a7b13ae9413095feffd9924566
```

**Scope**

```
./src/
    PasswordStore.sol
```

### Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

## Executive Summary

*Add some notes about how the audit went, types of things you found, etc.*

*We spent X hours with Z auditors using Y tools, etc*

### Issues found

| Severity | Number of issues founded |
|---|---|
| High | 2 |
| Medium | 0 |
| Low | 0 |
| Info | 1 |
| Total | 3 |

## Findings

### High

#### [H-1] Storing the password on-chain makes it visible to anyone, and no longer private.

**Description:** All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract.

We show one such method of reading any data off chain below.

**Impact:** Anyone can read the private password, severly breaking the functionality of the protocol.

**Proof of Concept:** (Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
make anvil
```

1. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool

We use 1 because that's the storage slof of `s_password` in the contract.

```
cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this:

`0x6d7950617373776f726400000000000000000000000000000000000000000014`

You can then parse that hex to a string with:

```
cast parse-bytes32-string 0
    x6d7950617373776f726400000000000000000000000000000000000000000014
```

And get an output of:

```
myPassword
```

**Recommended Mitigation:** Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remeber another password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

### [H-2] 'PasswordStore::setPassword' has no access controls, meaning a non-owner could change the password.

**Description:** The 'PasswordStore::setPassword' function is set to be an 'external' function, however, the natspec of the function and the overall purpose of the smart contract is that 'This function allos only the owner to set a new password'.

```
    function setPassword(string memory newPassword) external {
@>      // @audit: There are no access controls
        s_password = newPassword;
        emit SetNewPassword();
```

```
5       }
```

**Impact:** Anyone can set/change the password of the contract, severly breaking the contract intended functionality.

**Proof of Concept:** Add the following to the 'PasswordStore.t.sol' test file.

Code

```
1       function test_anyone_can_set_password(address randomAddress) public
            {
2           vm.assume(randomAddress != owner);
3           vm.prank(randomAddress);
4           string memory expectedPassword = "myNewPassword";
5           passwordStore.setPassword(expectedPassword);
6
7           vm.prank(owner);
8           string memory actualPassword = passwordStore.getPassword();
9           assertEq(actualPassword, expectedPassword);
10      }
```

**Recommended Mitigation:** Add an access control conditional to the 'setPassword' function.

```
1  if(msg.sender != owner) {
2      revert PasswordStore_NotOwner();
3  }
```

### Informational

#### [I-1] The 'PasswordStore::getPassword' natspec indicates a parameter that dosen't exist, causing the natspec to be incorrect.

**Description:**

```
1       /*
2        * @notice This function allows only the owner to set a new
             password.
3        * @param newPassword The new password to set.
4        */
5       // @audit-high: any user can set a password
6       function setPassword(string memory newPassword) external {
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natspec says it should be `getPassword(string)`.

**Impact:** The natspec is incorrect.

**Recommended Mitigation:** Remove the incorrect natspec line.

```
1 -     * @param newPassword The new password to set.
```